

Installing Drydock

Drydock is a Python command-line application. Installation places the **drydock** executable and its versioned application resources in a Python environment. It does not create a project, a Blueprint, or a workspace in the user's home directory.

This guide assumes the following are already true:

- Python 3.11 or later is available.
- **drydock** is installed and resolves on **PATH**.
- **\$PROJECTS** names an existing directory that holds the user's projects.

For example:

```
export PROJECTS="$HOME/projects"
mkdir -p "$PROJECTS"
drydock --version
```

Install the command

The PyPI distribution is named **drydock-sdd**; the installed command is named **drydock**.

Install it as an isolated command-line tool:

```
uv tool install drydock-sdd
```

Alternatively, use **pipx**:

```
pipx install drydock-sdd
```

For a Python virtual environment, install with pip:

```
python -m pip install drydock-sdd
```

Verify the installation:

```
drydock --version
drydock --help
```

`uv tool` and `pipx` install the command in an isolated Python environment and place a command wrapper on `PATH`. They are the recommended installation modes for an interactive CLI. `pip install` installs Drydock and its Python dependencies into the active virtual environment.

Configure the workspace

Use `$PROJECTS/drydock` as the Drydock workspace. Use `$PROJECTS` as the root where Drydock writes generated applications. Both directories must exist before they are configured.

```
mkdir -p "$PROJECTS/drydock"

drydock config set drydock_workspace "$PROJECTS/drydock"
drydock config set drydock_build_directory "$PROJECTS"
drydock config show
```

The resulting layout is:

```
$PROJECTS/
├── drydock/                # Drydock workspace
│   ├── targets/           # Created by `drydock init`
│   │   └── logs/          # Created when commands run
│   └── <Target>/          # Generated application output
```

Do not create `targets/` manually. `drydock init <Target>` creates it and creates the target baseline.

Create the first target

```
drydock init Example
drydock status
```

This creates the user-owned target workspace:

```
$PROJECTS/drydock/targets/Example/
├── blueprint/
│   └── sources/
├── evidence/
├── logs/
├── QuarterDeck/
│   └── console.yaml
└── METADATA.md
```

The target name is a logical identifier, not a path. It cannot contain path separators or ...

Persistent configuration and overrides

`drydock config set` writes per-user configuration; it does not modify the installed package. Its file location is:

Platform	Configuration file
Linux and macOS	<code>\$XDG_CONFIG_HOME/drydock/.env</code> , or <code>~/.config/drydock/.env</code> when <code>XDG_CONFIG_HOME</code> is unset
Windows	<code>%APPDATA%\drydock\.env</code>

Environment variables override saved configuration for a single command. This is appropriate for CI and for switching workspaces temporarily:

```
DRYDOCK_WORKSPACE="$PROJECTS/client-a/drydock" drydock status
```

The configurable values are:

CLI key	Environment variable	Purpose
<code>drydock_workspace</code>	<code>DRYDOCK_WORKSPACE</code>	Workspace containing <code>targets/</code> and workspace logs
<code>drydock_build_directory</code>	<code>DRYDOCK_BUILD_DIRECTORY</code>	Root containing generated application directories
<code>drydock_model</code>	<code>DRYDOCK_MODEL</code>	Model selection for supported LLM-assisted commands
<code>llm_provider</code>	<code>LLM_PROVIDER</code>	Subscription CLI provider: <code>claude</code> or <code>codex</code>
<code>prompt_warn_kb</code>	<code>PROMPT_WARN_KB</code>	Prompt-size warning threshold in KiB
<code>quarterdeck_port</code>	<code>QUARTERDECK_PORT</code>	Default QuarterDeck port

What installation provides

The `drydock-sdd` distribution installs:

- the `drydock` Python package and command-line entry point;
- Drydock's versioned Rigging rules and Blueprint templates;
- versioned prompt contracts;
- the QuarterDeck runtime template; and
- required Python runtime dependencies.

The installation does not create or own a user project. The following data is created under the configured workspace by Drydock commands and remains owned by the user:

- Targets and their Blueprints;
- manifests, evidence, Sea Trials, Soundings, and logs;
- QuarterDeck target configuration and state; and

- generated application code under `drydock_build_directory`.

The installed resources are coupled to the installed Drydock release. Upgrading `drydock-sdd` upgrades the command and its packaged rules, templates, prompts, and QuarterDeck runtime together. Existing targets remain in the configured workspace and are not rewritten by installation or upgrade.

Canonical product specification release status

The canonical Drydock product specification is intended to be a versioned, read-only resource of the Drydock release. It is distinct from a user's Blueprint.

The current wheel configuration packages Rigging, prompts, and QuarterDeck, but does **not** package `docs/Drydock_Specification.md`. An installed release therefore cannot yet load that specification from package resources. Until the wheel includes it and `paths.py` resolves it in installed mode, operations that require the product specification require a source checkout.

Release engineering must add the specification to the wheel and validate the installed-wheel path before documenting it as available to `drydock` users.

Upgrade and removal

Upgrade the CLI without touching user workspaces:

```
uv tool upgrade drydock-sdd
# or: pipx upgrade drydock-sdd
# or, in an active virtual environment: python -m pip install --upgrade drydock-sdd
```

Uninstalling the Python distribution removes the command and packaged resources only. It does not remove the user configuration file, `$PROJECTS/drydock`, or generated applications under `$PROJECTS`.

Release verification

Before publishing a release, build a clean wheel and inspect it:

```
python -m hatchling build
unzip -l dist/drydock_sdd-*.whl
```

The wheel must contain the `drydock` package, `drydock/resources/Rigging/`, `drydock/resources/prompts/`, `drydock/resources/QuarterDeck/`, and, once the canonical-specification release work is complete, `drydock/resources/docs/Drydock_Specification.md`.

Install the wheel into an isolated environment and run `drydock --version`, `drydock config show`, and `drydock init Example` against a temporary `$PROJECTS` directory. This validates the artifact a PyPI user receives rather than only the source checkout.